Pizzeria

COMPREHENSIVE PIZZA SALES ANALYSIS USING SQL

Presented by : Sriram Joshi

PROBLEM STATEMENT AND OBJECTIVES

A pizza chain has been collecting sales data from its multiple outlets to understand customer preferences and improve business performance. However, they face challenges in deriving actionable insights due to the lack of structured analysis. This project aims to analyze the pizza sales data, uncover patterns, and present key metrics to help the management make data-driven decisions for optimizing menu offerings, pricing strategies, and operational efficiency.

Objectie is to conduct an in-depth analysis of pizza sales data and develop a comprehensive dashboard that provides actionable insights into order trends, revenue generation, and customer preferences.

KEY STAKEHOLDERS

- Business Owners: Interested in revenue, popular menu items, and high-performing categories to make strategic decisions.
- Operations Manager: Needs insights into peak order times and distribution by category to streamline operations.
- Marketing Team: Wants to identify customer preferences to create targeted promotional campaigns.
- Supply Chain Team: Requires pizza size and type preferences to manage inventory efficiently.

RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED

Provides insights into customer demand trends

QUERIES

```sql
SELECT
    COUNT(order_id) AS total_order
FROM
    orders;
```

RESULT

	total_order
▶	21350

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES

Helps stakeholders gauge overall financial performance.

QUERIES

```
SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),
            2) AS total_sales
FROM
    order_details
        JOIN
    pizzas ON pizzas.pizza_id = order_details.pizza_id;
```

RESULT

	total_sales_revenue
▶	817860.05

IDENTIFY THE HIGHEST-PRICED PIZZA

Supports pricing strategy adjustments for premium offerings

QUERIES

```
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

RESULT

	name	price
▶	The Greek Pizza	35.95

IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED

Aids inventory management and production planning

QUERIES

```sql
SELECT order_details.quantity, count(order_details.order_details_id) from order_details
GROUP BY order_details.quantity;

-- Secondary
SELECT
    pizzas.size,
    COUNT(order_details.order_details_id) AS Order_Count
FROM
    pizzas
        JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY Order_Count DESC
LIMIT 1;
```

RESULT

	size	Order_Count
▶	L	18526

LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES

Highlights customer preferences for menu optimization

QUERIES

```
SELECT
    pizza_types.name, SUM(order_details.quantity) AS quantit
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_i
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

RESULT

	size	Order_Count
▶	L	18526

JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED

Offers insights into category performance for marketing and supply chain

QUERIES

```sql
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC;
```

RESULT

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY

Pinpoints peak hours for staffing and delivery efficiency

QUERIES

```sql
SELECT HOUR(order_time) AS hour, COUNT(order_id) AS order_count FROM orders
GROUP BY HOUR(order_time);
```

RESULT

hour	order_count
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399

JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS

Assists in understanding customer preferences by category

QUERIES

```
SELECT
    category, COUNT(name)
FROM
    pizza_types
GROUP BY category;
```

RESULT

category	COUNT(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY

Provides metrics for daily operational benchmarks

QUERIES

```sql
SELECT
    ROUND(AVG(Quantity), 0) AS Average_Ordered_Pizza
FROM
    (SELECT
        orders.order_date, SUM(order_details.quantity) AS Quantity
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date)
    AS order_quantity;
```

RESULT

Average_Ordered_Pizza
138

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE

Identifies high-revenue-generating items for focused marketing

QUERIES

```
SELECT
    pizza_types.name,
    ROUND(SUM(order_details.quantity * pizzas.price),
            2) AS Revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY Revenue DESC
LIMIT 3;
```

RESULT

name	Revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE

Highlights the revenue share of individual pizzas for better decision-making

QUERIES

```sql
SELECT
    pizza_types.category as Category,
    round(SUM(order_details.quantity * pizzas.price) /
    (SELECT round(SUM(order_details.quantity * pizzas.price),2) as Total_Sales
FROM order_details
        JOIN
pizzas ON order_details.pizza_id = pizzas.pizza_id) * 100,2) as Revenue
FROM pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY Revenue DESC;
```

RESULT

Category	Revenue
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME

Tracks revenue growth trends for strategic planning

QUERIES

```
SELECT order_date,
sum(Revenue) over(order by order_date) as Cumulative_Revenue
FROM
(SELECT
    orders.order_date,
    SUM(order_details.quantity * pizzas.price) AS Revenue
FROM
    order_details
        JOIN
    pizzas ON order_details.pizza_id = pizzas.pizza_id
        JOIN
    orders ON orders.order_id = order_details.order_id
GROUP BY orders.order_date) as Sales;
```

RESULT

order_date	Cumulative_Revenue
2015-01-01	2713.8500000000004
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6
2015-01-05	11929.55
2015-01-06	14358.5
2015-01-07	16560.7
2015-01-08	19399.05
2015-01-09	21526.4
2015-01-10	23990.350000000002
2015-01-11	25862.65
2015-01-12	27781.7
2015-01-13	29831.300000000003

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY

Provides insights into category leaders to optimize product offerings

RESULT

category	name	Revenue
Chicken	The Thai Chicken Pizza	43434.25
Chicken	The Barbecue Chicken Pizza	42768
Chicken	The California Chicken Pizza	41409.5
Classic	The Classic Deluxe Pizza	38180.5
Classic	The Hawaiian Pizza	32273.25
Classic	The Pepperoni Pizza	30161.75
Supreme	The Spicy Italian Pizza	34831.25
Supreme	The Italian Supreme Pizza	33476.75
Supreme	The Sicilian Pizza	30940.5
Veggie	The Four Cheese Pizza	32265.70000000065
Veggie	The Mexicana Pizza	26780.75
Veggie	The Five Cheese Pizza	26066.5

QUERIES

```
SELECT category, name, Revenue
FROM
(SELECT category, name, Revenue,
rank() OVER( PARTITION BY category ORDER BY Revenue DESC) AS rn
FROM
(SELECT pizza_types.category, pizza_types.name,
sum(order_details.quantity * pizzas.price) AS Revenue
FROM pizza_types
JOIN pizzas
ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details
ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category, pizza_types.name
ORDER BY Revenue DESC) as a) AS b
WHERE rn <= 3;
```

THANK YOU FOR YOUR ATTENTION